

Cambia BigQuery Ingestion Design

Status: DRAFT for review. Tracked by XPORT-2596. Several data-contract questions are still open with Cambia; the design states its assumptions explicitly rather than waiting on them.

1. Goal

This service ingests Prime/Judi pharmacy-claims data from Cambia's Google BigQuery project into Abacus. It runs as a scheduled Kubernetes CronJob on EKS, queries BigQuery cross-cloud, and lands the data in S3.

Scope:

- Ingest Cambia pharmacy-claims data from BigQuery only.
- Support initial full load, daily incremental load, periodic full refresh, and replay.
- Authenticate with AWS-to-GCP Workload Identity Federation and service-account impersonation. **No long-lived credential is stored anywhere.**
- Handle failure recovery, idempotent writes, logging, alerting, and PHI/PII compliance requirements.

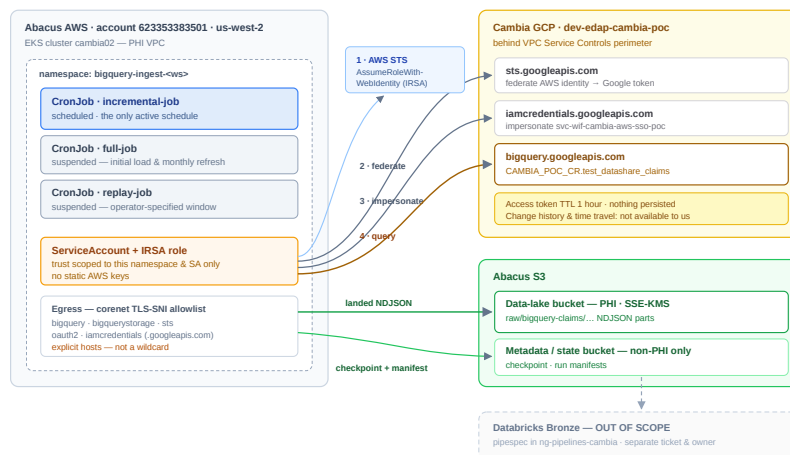
This service ends at S3. Loading into the Databricks Bronze layer — and the merge, dedup and delete handling that goes with it — is explicitly out of scope. Bronze loading is a Databricks workflow defined as a pipespec in the per-customer connector repo (`ng-pipelines-cambia`), which the platform clones into Databricks; it cannot live in this service's repo. Every other API connector on the platform follows the same split: the connector lands raw output in S3, and a Databricks workflow loads Bronze from there. §7 and §8 define the output contract precisely, because that contract is the handoff between the two pieces of work.

Pharmacy-claims data is treated as PHI/PII. Design, implementation, and release must go through security/compliance review.

2. Overall Architecture

Cambia BigQuery Ingestion — Overall Architecture

No inbound traffic - no stored credential - service boundary ends at S3



Key resources:

- **EKS CronJob:** runs daily incremental loads; full loads, periodic full refreshes and replays are triggered through controlled manual jobs; concurrency must be disabled and a runtime timeout must be set.
- **IRSA:** the pod uses a dedicated ServiceAccount and a least-privilege IAM Role.

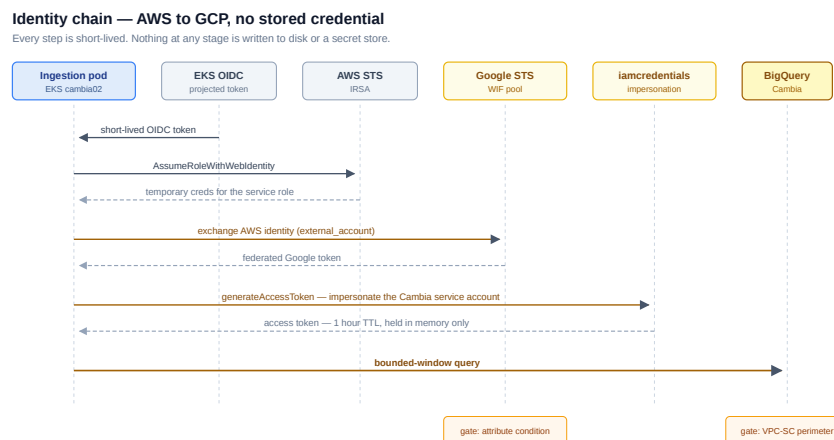
- **No secret store.** Unlike other ingestion services, this one holds no long-lived credential — see §3.
- **S3 data lake bucket:** stores raw claims payload; must follow PHI storage encryption and audit requirements.
- **S3 metadata/state bucket:** stores checkpoints, manifests, and run summaries; only non-PHI metadata is allowed.
- **Egress:** outbound access goes to five Google endpoints permitted by the centralized corenet TLS-SNI allowlist — `bigquery`, `bigquerystorage`, `sts`, `oauth2`, `iamcredentials`, all `.googleapis.com`. All AWS service traffic stays on VPC endpoints.

No Ingress, ALB, or NLB is created. The service has no inbound application traffic.

3. Authentication

Cambia exposes BigQuery through Workload Identity Federation. **There is no client secret, no API key, and no service-account key file** — nothing to store, nothing to rotate, and nothing that keeps working if it leaks.

3.1 Identity chain



Cambia's provider validates our AWS role ARN against an attribute condition; their VPC Service Controls perimeter separately permits the impersonated service account. Both are configured and verified working (XPORT-2524).

3.2 Two implementation details that are not optional

Both cost significant debugging time during the POC and must survive into the implementation:

- **IRSA credentials must be resolved and exported explicitly** before the Google auth library initialises. Its AWS credential source reads `AWS_*` environment variables first and otherwise falls back to EC2 IMDS — which signs as the **node group role**, not the pod's role. That identity is not trusted by Cambia's attribute condition, and the resulting failure looks like a trust-configuration problem on their side rather than a bug on ours.
- **The GCP project must be passed explicitly.** `google.auth.load_credentials_from_file` calls `get_project_id()`, which reaches `cloudresourcemanager.googleapis.com` — a host the egress allowlist does **not** include. It hangs for the full timeout and then fails even when the token exchange itself succeeded. Construct the credentials directly instead.

3.3 IAM role

The POC borrowed `cambia02-airbyte-dev`, whose trust policy admits any ServiceAccount in the `airbyte-dev` namespace. Production requires a dedicated role whose trust policy is scoped to this service's namespace and ServiceAccount only.

This is a dependency on Cambia: they must add the new role ARN to their provider's attribute condition. Note their condition matches the STS form `arn:aws:sts::<acct>:assumed-role/<role>`, not the IAM form `arn:aws:iam::<acct>:role/<role>`.

4. Data Pull Modes

4.1 Initial Full Load

Reads the entire source table and lands it under the `full/` prefix. Required once per workspace before incremental can run, because incremental fails closed without a checkpoint (§4.2).

After the full load completes, the service records the maximum observed watermark value as the initial high-watermark. A full load must not run concurrently with an incremental load for the same workspace.

Production volumes are unknown (§12). The full-load path must therefore be **chunkable by watermark range and resumable**, rather than assuming a single unbounded query completes inside the job deadline.

4.2 Daily Incremental Load

```
1 SELECT *
2 FROM   `
```

`window_start` is the previous successful checkpoint's `last_successful_high_watermark`. `window_end` is fixed at run start before any query is issued. The lower bound is exclusive so records already covered are not reselected; the upper bound is inclusive so the run processes a closed, auditable window.

The watermark column is configuration, not a code constant — `received_at` is the assumed default (§12, A1).

Cost note. The source table is partitioned by DAY on `associated_prescription_service_date`, a business date, while the watermark is `received_at`. A filter on the watermark therefore **cannot prune partitions** and each incremental run scans the full table. Cambia pays for that compute. Once production volumes are known this should be raised with them — either partitioning on the watermark column, or an agreed compound filter that does prune, would materially reduce it.

Timezone. `received_at` is a `DATETIME` and carries no timezone. Window bounds must be computed in whatever zone the source populates, not blindly in UTC. Until Cambia confirms, the zone is a configuration value with **no safe default**. Getting it wrong drops or duplicates rows at every window boundary.

Why incremental does not automatically run a full load when the checkpoint is missing

If the incremental run cannot find a checkpoint, the job must fail and alert. It must not silently switch to a full load.

A missing checkpoint does not necessarily mean this is the first run. It may indicate an incorrect state bucket or prefix, an IAM/KMS permission issue, a transient S3 read failure, incorrect workspace configuration, or a deleted or corrupted checkpoint. Automatically running a full load in that state would turn a configuration problem into a large PHI re-pull against a customer's BigQuery — at their expense — while hiding the real failure and corrupting the production high-watermark.

First-time initialization must be triggered explicitly through `full` mode.

4.3 Periodic Full Refresh

A scheduled full reload, initially monthly, in addition to the daily incremental.

This is a correctness requirement, not an optimisation. The reasoning is in §8.5 and should be read before anyone proposes removing it.

4.4 Replay

Re-pulls data for an operator-specified historical window. Used for backfills, fixes, audit reconciliation, or disaster recovery.

Replay writes to a separate `replay/` prefix under its own `run_id` and **does not modify the production checkpoint**.

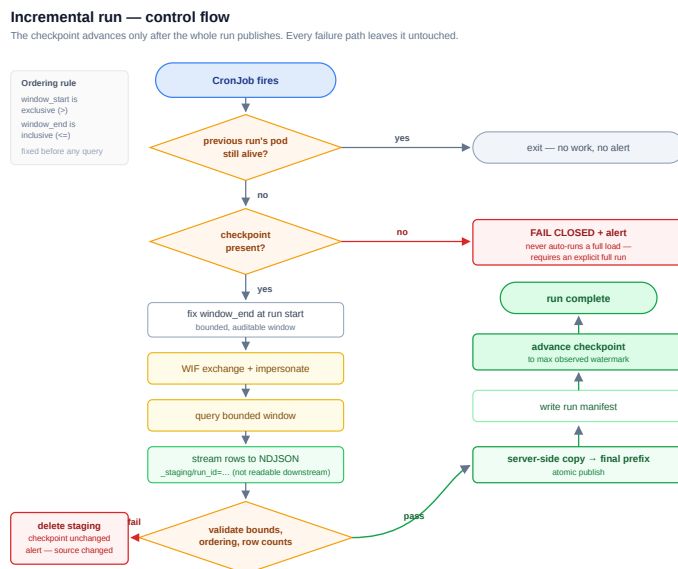
Replayed data is landed but not automatically consumed; promoting it downstream is a deliberate, ticketed operation.

5. Service Components And Main Flow

5.1 Service Components

- `IngestionRunner` — CronJob entry point; generates `run_id`, loads configuration, dispatches to `full`, `incremental`, `refresh`, or `replay`.
- `GcpCredentialProvider` — resolves IRSA credentials, performs the WIF exchange and impersonation, holds the resulting token in process memory only.
- `BigQueryClient` — builds the bounded window query, executes the job, streams the result set, and surfaces the BigQuery job id for audit.
- `LandingWriter` — serialises rows to NDJSON, writes parts to the run-scoped staging prefix, computes content hashes.
- `CheckpointStore` — reads and writes checkpoints and manifests; stores only non-PHI metadata.
- `AlertPublisher` — sends safe summary alerts; never payload, member identifier, claim identifier, or credentials.

5.2 Incremental Main Flow



6. Checkpoint And Recovery

The checkpoint is stored in the metadata/state S3 bucket and contains only non-PHI metadata.

```
1 | s3://<state-bucket>/metadata/bigquery-claims/checkpoint/bigquery-claims-<ws>.json
```

Core fields:

- `run_id`
- `status` — `RUNNING`, `COMPLETED`, or `FAILED`
- `mode` — `full`, `incremental`, `refresh`, or `replay`
- `started_at` / `completed_at`
- `last_successful_high_watermark`
- `pending_high_watermark`
- `watermark_column` and `watermark_timezone` — recorded so a configuration change is visible in the state itself
- `landing_manifest_path` / `failed_manifest_path`
- `pod_name` / `pod_namespace`

Rules:

- The checkpoint is updated only after S3 payload and manifest writes succeed.
- `last_successful_high_watermark` advances only after the entire run succeeds.
- Incremental mode fails closed when no checkpoint is found; it must not switch to a full load.
- An initial checkpoint may be created only through explicit `full` mode.
- On failure, the previous successful high-watermark is kept; the next run re-covers the same window.
- If a pod crashes or a node is interrupted, the next run first checks whether the previous run still has an active pod. If it does, the new run exits. If it does not, the previous run is marked failed and recovery proceeds.

7. Manifest Design

The manifest is the non-PHI landing ledger for each run, stored in the metadata/state bucket. It never stores payload, claim identifier, member identifier, or query text containing predicates over PHI.

Purpose: audit which objects a run actually wrote; identify which parts landed after a failure; detect duplicate writes; and give the downstream Bronze workflow an authoritative row count and per-part checksum to reconcile against.

That last purpose matters more here than it does for other connectors, because **we have no source-side control totals** — Cambia does not supply expected row counts or hashes with the data (§12). The manifest is the only completeness evidence in the chain.

Recommended fields:

- `run_id`, `mode`, `status` — `LANDED` or `FAILED`
- `sequence_number` — landed part sequence within the run
- `s3_object_key` — must not contain sensitive identifiers
- `record_count`, `byte_size`, `content_sha256`
- `min_watermark` / `max_watermark` — watermark boundaries of records in the object
- `bigquery_job_id` — links the landed object back to a query job on Cambia's side, which is valuable during joint troubleshooting
- `error_category`, `http_status` — failures only
- `created_at`

8. Output Contract And Incremental Completeness

8.1 Landing layout

```
1 | s3://<data-lake-bucket>/raw/bigquery-claims/  
2 |   _staging/run_id=<run_id>/part-<sequence>.ndjson          ← in flight,  
   | never read downstream  
3 |   full/run_id=<run_id>/part-<sequence>.ndjson  
4 |   incremental/date=<yyyy-mm-dd>/run_id=<run_id>/part-<sequence>.ndjson  
5 |   refresh/date=<yyyy-mm-dd>/run_id=<run_id>/part-<sequence>.ndjson  
6 |   replay/run_id=<run_id>/part-<sequence>.ndjson
```

8.2 Atomic publish

Every part is written to the run-scoped `_staging/` prefix and server-side copied to its final prefix only after the whole run succeeds. On failure the staging prefix is deleted.

Nothing partially written is ever visible at a downstream-readable path. This keeps downstream consumption simple: the Bronze workflow discovers files by prefix and does not need to distinguish a complete incremental from a half-written one.

8.3 Format

- **NDJSON**, one JSON object per line, UTF-8.
- Uncompressed by default, matching `lucet`. Revisit once production volumes are known — `gzip` is a small change and likely worthwhile at claims scale.
- Type mapping: `DATE` / `DATETIME` / `TIMESTAMP` → ISO 8601 strings; `NUMERIC` / `BIGNUMERIC` → JSON number; `BYTES` → base64. Nested and repeated fields are preserved as JSON objects and arrays.
- Part files roll at a configurable size so no single object becomes unwieldy.

8.4 Completeness controls per run

- Validate every returned row satisfies `window_start < watermark <= window_end`.
- Validate watermark values are non-decreasing, since the query is sorted.
- Validate the row count written to S3 equals the row count read from BigQuery.
- Record `record_count`, `sequence_number`, `min_watermark`, `max_watermark` and `content_sha256` per part in the manifest.
- Advance the checkpoint only after all parts are landed, copied, and recorded.
- If at least one row was returned, set the next checkpoint to the **maximum observed watermark**, not to `window_end`.
- If no rows were returned, set the checkpoint to `window_end` only after the query completed successfully.
- If any query, S3 write, manifest write, or validation step fails, do not advance the checkpoint.

8.5 Why the periodic full refresh is a correctness requirement

BigQuery has native change tracking, and it is **not available to us**. Verified against Cambia dev:

- `CHANGES()` returns *"Change history is not enabled for table"*. The `enable_change_history` option defaults to `FALSE`, is documented as *In preview*, is not retroactive, and requires `bigquery.tables.update` — which our `dataViewer` grant does not include. Only Cambia can enable it.
- Time travel is denied entirely. `FOR SYSTEM_TIME AS OF` fails with `Not found` at one minute, two hours, forty-eight hours and six days, while an ordinary `SELECT COUNT(*)` on the same table succeeds. This is a deliberate restriction on historical versions, not a window expiry.
- Even if enabled, change history is bounded by the time travel window — seven days at most, and Cambia can shorten it to two without telling us — capped at a one-day range per call, silently disqualified by multi-statement transactions in the window, and requires `bigquery.admin` if the table ever had row-level access policies.

The consequence is concrete: **we have no way to independently verify that the watermark column is trustworthy**. If a restated claim's watermark does not move, that row is lost silently and permanently, with nothing in our logs to indicate it happened.

The periodic full refresh is the only mechanism that recovers such rows. It should not be removed as redundant unless the watermark question (§12, A1) has first been settled with Cambia. Source-side control totals would strengthen this further but require Cambia to change their process.

This is not a tooling limitation. Airbyte's BigQuery source connector documents *"Change Data Capture: No"* and performs the same cursor-based extraction; Databricks Lakehouse Federation requires a downloaded service-account key and is ruled out on those grounds (XPORT-2578). Every available option lands on the same watermark strategy.

9. Error Handling

Default rule: **failures do not advance the successful high-watermark**.

During a run, payload files are written to a run-scoped staging prefix, and the final downstream-readable prefix is updated only after the whole run succeeds.

9.1 Common failures

Failure	Behaviour
WIF exchange or impersonation fails	Fail the job; do not query BigQuery
BigQuery query fails or times out	Bounded exponential backoff; on exhaustion fail the job, checkpoint unchanged
S3 staging or manifest write fails	Fail the job, checkpoint unchanged
Validation failure (§8.4)	Fail before publish and alert — this indicates a source-semantics change, not a transient fault
Missing or corrupt checkpoint on incremental	Fail closed; requires a manual <code>full</code> run
Previous run still alive	Exit without doing work
Checkpoint write failure	Fail the job and alert for manual handling
Zero rows in window	Success ; checkpoint advances to <code>window_end</code>

That last row deserves attention. A silently broken upstream looks identical to a genuinely quiet day, forever. A **"no rows landed in N consecutive runs"** alert is the only thing that distinguishes them, and N should be configurable per workspace because the right value depends on the feed's cadence.

There is no dead-letter queue. The service is pull-based and its recovery model is "the next run re-covers the window", which the checkpoint semantics make safe.

9.2 Handled failure

1. Mark the run `FAILED` in checkpoint/run metadata.
2. Write a safe failure summary to the failed manifest.
3. Delete all files under the current run's staging prefix.
4. Publish nothing to the final prefix.
5. Do not initialize or advance the production checkpoint.
6. Exit non-zero so CronJob status and alerting reflect the failure.

9.3 Pod crash or unexpected exit

The application may not get a chance to mark the run failed or clean up staging files. The next run therefore:

1. Checks whether the previous pod still exists. If it does, exits to avoid concurrent processing.
2. If it does not, marks the old run `FAILED`.
3. Deletes the old run's staging prefix before starting.
4. Re-runs from the previous successful high-watermark with a new `run_id`.
5. Never publishes old staging files, and never initializes the checkpoint from an incomplete run.

10. Logging And Alerting

Allowed log fields: `run_id`, workspace/environment, mode, window bounds, row count, bytes written, BigQuery job id, retry count, duration, sanitized error category.

Forbidden: row values of any kind, claim number, member identifier, prescriber identifier, drug name, query text containing PHI predicates, access tokens, and raw exception details that may embed any of the above.

SDK and lower-level HTTP debug logging must be disabled or centrally sanitized so that headers, query strings, and bodies are not logged automatically.

Alerts worth having:

- run failed N consecutive times
- **no rows landed in N consecutive runs** (§9.1)
- run did not fire at all when scheduled — a skipped run produces no logs, so failure and absence are different signals and only one of them is visible
- validation failure (§8.4), raised separately because it means the source changed rather than that something broke

11. Security And Permission Decisions

- **No long-lived credential exists.** Identity is proven per request and the resulting token expires in one hour. This is the property that ruled out both Airbyte and Databricks Lakehouse Federation, each of which requires a static GCP service-account key — a non-expiring, portable credential to PHI. It should be treated as a design constraint rather than an implementation detail.
- The IRSA trust policy must be scoped to the specific namespace and ServiceAccount. Static AWS keys and broad node-role permissions are not allowed.
- The runtime IAM Role may only write its own S3 prefixes, decrypt the specific KMS keys, and read required SSM parameters. It must not have `s3:*`, `kms:*`, or wildcard permissions.
- Landed claims data is PHI and goes to the PHI data-lake bucket under its existing CMK and access controls. Operational state — checkpoints and manifests — is non-PHI and belongs in the metadata/state bucket. The two must not be mixed.
- All S3 buckets must enable SSE-KMS, Block Public Access, TLS enforcement, access auditing, and appropriate versioning/lifecycle policies.
- The pod exposes no inbound traffic. Outbound access is restricted by the centralized corenet TLS-SNI allowlist to the five Google endpoints in §2; all AWS service traffic stays on VPC endpoints.
- Note the allowlist is a list of explicit hosts, not a `*.googleapis.com` wildcard. Any future dependency on another Google API — `cloudresourcemanager` being the one already encountered — requires a new SecOps request.

12. Open Assumptions

The data-contract questions below are open with Cambia via the Product Manager. This design does not wait on them; it states each assumption and what changes if the assumption proves wrong.

#	Assumption	If wrong
A1	Incremental keys on a watermark column, assumed <code>received_at</code>	If <code>received_at</code> is set only at first insert and never updated on restatement, incremental alone is unsafe and the periodic full refresh (§4.3) becomes the primary correctness mechanism rather than a safety net

A2	Natural key is (<code>claim_number</code> , <code>claim_sequence_number</code>) — all 20 POC rows were distinct on that pair	If production needs a line or version field, the Bronze merge keys change. Affects the downstream workflow more than this service
A3	Claims may be restated after first delivery	If the feed proves append-only, the downstream load can be simplified. The reverse is far more expensive to correct, so the design assumes the harder case
A4	Each incremental run scans the full table	Partition/watermark mismatch, §4.2. Sizing and cost estimates assume a full scan until production volumes are known
A5	One logical stream — one table	If production is many tables, the checkpoint becomes per-table and §6 changes shape

13. Appendix — Verified Reference Values (cambia02 dev)

AWS account / region	623353383501 / us-west-2
EKS cluster	cambia02
Data-lake bucket	abacus-cambia02-data-lake-dev-623353383501-us-west-2
GCP project / number	dev-edap-cambia-poc / 982051858538
WIF pool / provider	aws-cambia-sso-poc-pool / aws-cambia-sso-poc-provider
Impersonated service account	svc-wif-cambia-aws-sso-poc@dev-edap-cambia-poc.iam.gserviceaccount.com
POC table	CAMBIA_POC_CR.test_datashare_claims — 20 rows, 35 columns
Table layout	partitioned DAY on <code>associated_prescription_service_date</code> ; clustered on <code>carrier_id</code> , <code>claim_status_code</code> , <code>external_group_id</code> ,

	<code>submitted_product_service_id</code>
Dataset time travel	<code>max_time_travel_hours = 168</code> — configured, but historical reads are denied to us
Egress allowlist	<code>bigquery</code> , <code>bigquerystorage</code> , <code>sts</code> , <code>oauth2</code> , <code>iamcredentials</code> — all <code>.googleapis.com</code> . Not <code>cloudresourcemanager</code>